

Assignment 6 Report

GUI Multi Client Chat Application using Python Tkinter and TCP Sockets

Submitted By

Name: Niharjyoti Choudhury
Roll Number: CSB24011

1. Objective

The objective of this assignment is to develop a GUI-based Multi Client Chat Application using Python. The application demonstrates GUI programming, event-driven programming, multithreading, socket programming, and client-server communication. The project also aims to provide a user-friendly interface for multiple clients communicating over a TCP network.

2. Introduction

A client-server chat application enables multiple users to communicate over a network using a central server. Unlike command-line chat applications, GUI-based applications provide a more interactive and user-friendly experience. The project uses Python's Tkinter library to design the graphical interface and Python socket programming for communication between clients and the server.

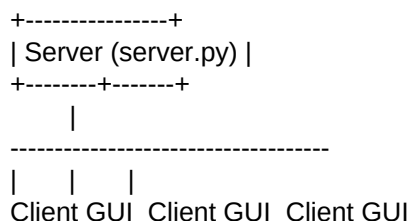
3. Software Requirements

- Ubuntu 24.04 LTS
- Python 3
- Tkinter
- Mininet
- Git
- GitHub
- Wireshark

4. System Architecture

The system follows the Client-Server Architecture.

- One server accepts incoming client connections.
- Multiple clients connect using TCP sockets.
- The server broadcasts messages.
- The server manages online users.
- The server stores chat history and performance data.



5. GUI Design

The client interface contains Login Window, Server IP Entry, Username Entry, Connect Button, Chat Window, Chat Area, Message Entry, Send Button, Disconnect Button, Online Users List, and Status Label.

Tkinter widgets used: Frame, Label, Entry, Button, Listbox, ScrolledText, MessageBox.

6. Features Implemented

GUI Login Screen
Multi-client Communication
Broadcast Messaging
Private Messaging
Online Users List
Connect/Disconnect
Multithreading
CSV Chat History
Performance Logging
Event-driven Programming

7. Working Procedure

1. Start the server.
2. Launch multiple GUI clients.
3. Enter the server IP and username.
4. Connect to the server.
5. Send broadcast messages.
6. Send private messages using: /msg username message.
7. Disconnect from the server.

8. Files Used

server.py
client_gui.py
chat_history.csv
performance_results.csv
README.md

9. Testing

The application was tested with multiple clients using Mininet. Verified: Successful Connection, Broadcast Messaging, Private Messaging, Online User Updates, Client Disconnection, Chat History Logging, and Performance Data Generation.

10. Screenshots

Include screenshots of: Login Window, Connected Clients, Broadcast Message, Private Message, Online Users List, Server Terminal, and Wireshark Capture (if required).

11. Learning Outcomes

GUI Programming using Tkinter
Socket Programming
TCP Communication
Event-driven Programming
Multithreading
Network Programming
Client-Server Architecture
File Handling
Git and GitHub

12. Conclusion

The GUI Multi Client Chat Application was successfully implemented using Python and Tkinter. The application supports multiple clients, broadcast messaging, private messaging, online user management, and chat history logging. Multithreading ensures the GUI remains responsive while handling network communication. The project successfully demonstrates the concepts of GUI programming, socket programming, event-driven programming, and client-server communication.

Submitted By

Niharjyoti Choudhury

Roll Number: CSB24011